



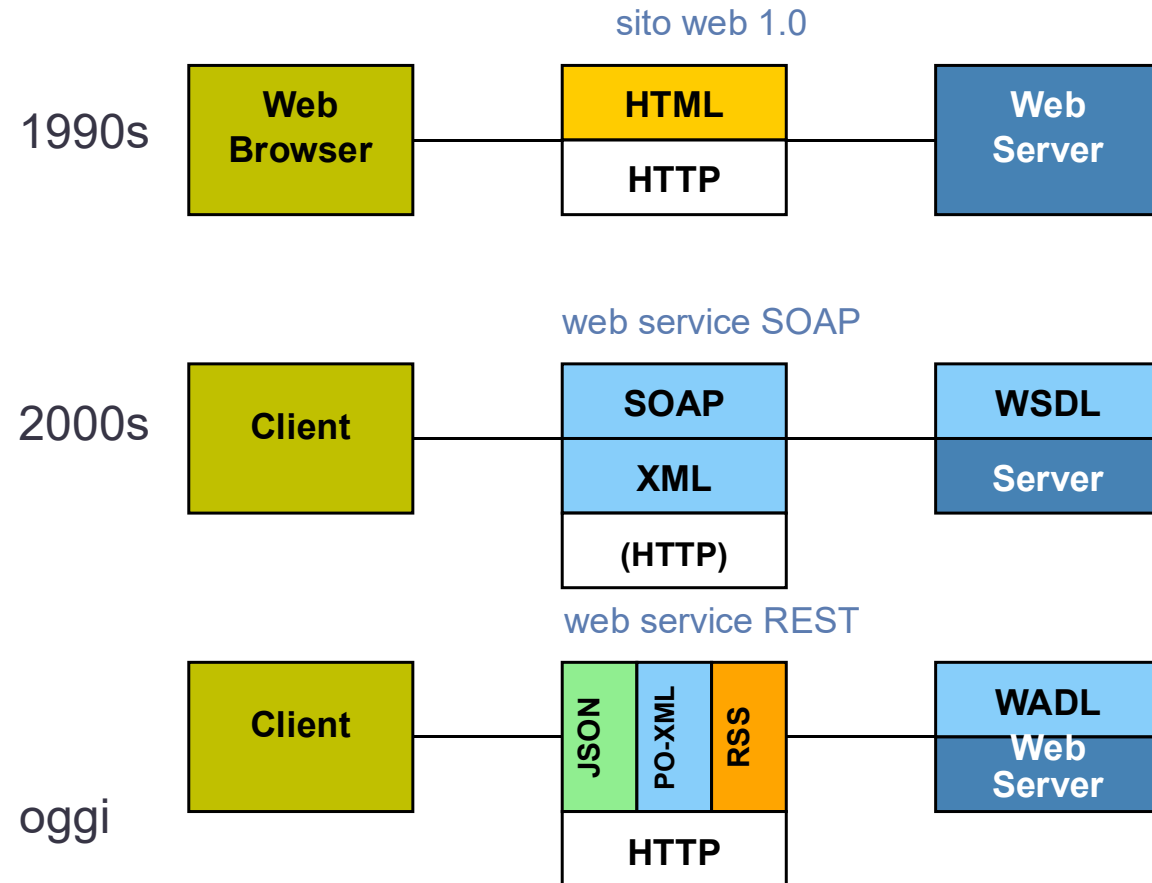
Services Computing REST

Prof. Michele Ciavotta
Corso di Sistemi Distribuiti

Sezione 1

DAL WEB AI SERVIZI RESOURCE-ORIENTED

Dal Web Statico a REST: l'evoluzione dei Servizi Web



- **Web 1.0 (HTML + HTTP):** inizialmente solo HTML e HTTP → i servizi erano limitati alla fornitura di pagine web.
- **SOAP (XML + WSDL):** i servizi risultano fortemente caratterizzati e descritti con elevata precisione. Possono persino essere "**compilati**". Tuttavia, i servizi sono complessi e difficili da descrivere e comprendere.
- **REST (HTTP + JSON/XML):** si **torna ai principi del protocollo HTTP** eliminando le ridondanze e assegnando una semantica ai verbi (GET, POST, PUT...) e agli URI.
 - Semplicità, scalabilità, e aderenza al Web lo rendono lo standard di fatto per le API moderne.

Architetture basate sulle risorse

URI sta per *Uniform Resource Identifier*. È una stringa di caratteri che identifica in modo univoco una risorsa su Internet (come una pagina web, un file o un'immagine).

È possibile considerare un sistema distribuito come una **raccolta di risorse gestite individualmente dai componenti**.

Le risorse possono essere **aggiunte, rimosse, recuperate o modificate** da applicazioni (remote).

REST (Representational State Transfer) vede il sistema distribuito come una raccolta di risorse identificate univocamente tramite un singolo schema di naming (URI).

L'interfaccia uniforme: RESTful prevede quattro operazioni HTTP:

1. PUT (modifica/crea),
2. POST (crea nuova risorsa),
3. GET (recupera stato),
4. DELETE (elimina risorsa).

I messaggi REST sono completamente auto-descrittivi: contengono tutti i metadati necessari per la loro interpretazione, senza fare affidamento su contesto di sessione esterno.

L'esecuzione è stateless: dopo ogni operazione il componente dimentica tutto del chiamante, rendendo le operazioni idempotenti e semplificando la scalabilità del sistema.

Operazione	Descrizione
PUT	Modifica una risorsa / Nuovo stato
POST	Crea una nuova risorsa
GET	Recupera lo stato di una risorsa
DELETE	Elimina una risorsa

Esempio: Amazon S3 (Buckets e Objects).

Semplificare il modello condiviso

I sistemi distribuiti per funzionare devono essere basati su un [modello condiviso](#)

- Con WSDL/SOAP i sistemi SOA devono concordare un API comune e messaggi.
- Molto ben definito ma risulta eccessivamente complesso

REST è costruito intorno all'idea di [semplificare l'accordo](#) (attraverso defaults)

1. **nouns** sono necessari per nominare le *risorse di cui* si può parlare (scambiate, cancellate, create)
2. **verbs** sono le operazioni che possono essere applicate alle risorse nominate.
3. **content types** (ad esempio, **application/json**) definiscono quali rappresentazioni delle informazioni sono disponibili

Sezione 2

I PRINCIPI REST

Principi REST

Una **risorsa** è un'entità identificabile e accessibile, che rappresenta un oggetto concettuale del sistema (ad esempio: un utente, un documento, un ordine).

Una risorsa è una **rappresentazione di dati**, non oggetti o funzioni;

REST = **RE**presentational **S**tate **T**ransfer

- REST è uno **stile architettonico** per i sistemi distribuiti

1. Le risorse sono **identificate da URI** (es. **/baseURL/resourceType/{id}**).
2. Le risorse vengono manipolate attraverso le loro **rappresentazioni**
 - Possono esistere **più rappresentazioni** di una risorsa (ad esempio, XML, JSON, HTML).
3. **Comunicazione** tramite **messaggi**, che sono **autodescrittivi** e **senza stato**.
4. **Lo stato dell'applicazione** evolve per mezzo della **manipolazione** delle risorse da parte del client
5. **Hypermedia** è il modo per che ha il server per **guidare il comportamento del client**
 - **Links**

Path parameter

Introdotta nel 2000 nella tesi di dottorato di *Roy Fielding*, uno dei principali autori delle specifiche del protocollo HyperText Transfer Protocol (HTTP)

I principi di REST adottano i componenti architettonici del Web: URI E HTTP

Risorse e interfaccia uniforme

La risorsa

- Una risorsa è una qualsiasi informazione che può essere nominata: documenti, immagini, servizi, collezioni, ecc.

Le risorse e lo stato dell'applicazione

- Le risorse di solito cambiano con l'interazione con il client

Le risorse hanno **identificatori** (**vincolo**) (*unimib.it/{matricola}/pianoStudio*)

- `{matricola}` è chiamato *path parameter*

→ Una risorsa è qualsiasi cosa abbastanza importante da essere referenziata.

Le risorse espongono un **interfaccia uniforme** (**vincolo**)

→ GET, POST, PUT, DELETE + Identificatori nell'URI

- Architettura del sistema semplificata; visibilità migliorata.
- Semplifica l'evolubilità delle implementazioni.

I «vincoli»

sono i principi di progettazione di REST: regole architetturali che un servizio deve rispettare per dirsi RESTful.

Manipolazione di risorse attraverso rappresentazioni

- Su richiesta, un servizio può trasferire una rappresentazione della risorsa a un client.
 - Richiede **un'architettura client-server** (**vincolo**)
 - Ad esempio, l'elenco degli esami registrati per un certo id studente.
- Un client può trasferire al server una rappresentazione *modificata* di una risorsa
 - **Manipolazione di risorse attraverso rappresentazioni** (**vincolo**) - nessuna invocazione di azioni
 - Ad esempio, uno studente può modificare l'elenco degli esami aggiungendo una nuova registrazione all'elenco.
- Le rappresentazioni restituite dal server possono collegarsi ad altre rappresentazioni. I client possono seguire un collegamento proposto e ricevere o modificare le risorse.
 - **Hypermedia come motore dello stato dell'applicazione** (**vincolo**)
 - Ad esempio, accanto all'elenco degli esami registrati, può essere presente un link per aggiungere o eliminare una registrazione.

Statelessness e Cacheability

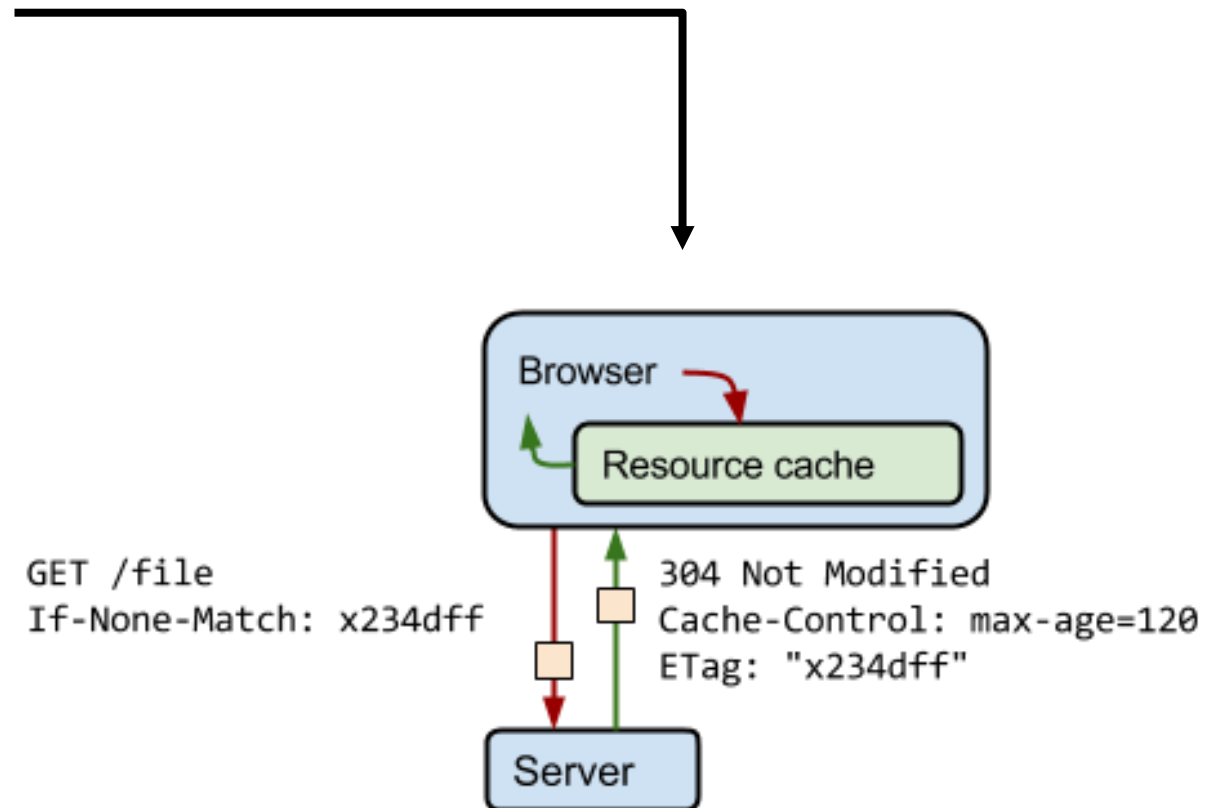
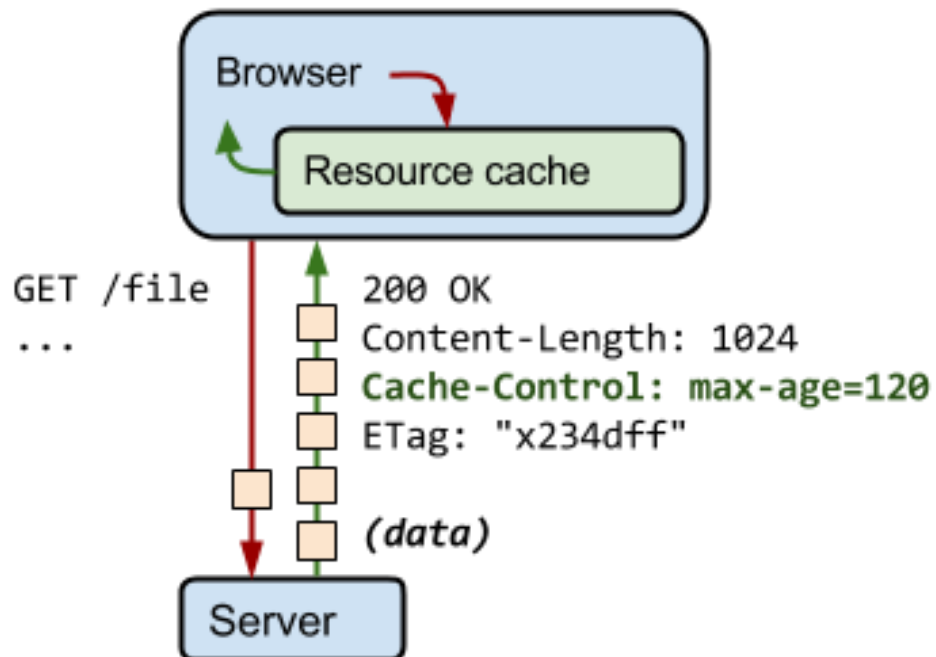
- **Interazioni prive di stato (vincolo)**
 - Ogni richiesta dal client al server deve contenere tutte le informazioni necessarie a comprendere la richiesta e non può sfruttare alcun contesto memorizzato sul server.
- La mancanza di stato richiede **messaggi autodescrittivi (vincolo)**
 - Tipi di media standard (ad esempio, application/json)
 - Meta-dati e dati di controllo
- Interfaccia uniforme + Mancanza di stato + Messaggi autodescrittivi = **Cacheability (vincolo)**
- **Cacheability** è una proprietà fondamentale delle architetture REST e indica la **capacità delle risposte HTTP di essere memorizzate in cache** da client, proxy, gateway, o CDN, al fine di migliorare l'efficienza, ridurre il carico sui server e diminuire la latenza percepita.

Ulteriori informazioni sulla cache

- La **cacheability** richiede un sistema a livelli (**vincolo**)
 - Almeno un livello di cache
 - Nella cache è memorizzata l'ultima rappresentazione di una risorsa
 - Una cache viene invalidata quando viene modificato lo stato della risorsa (PUT, DELETE).
- La cache **riduce la latenza e il traffico di rete**
- Vengono memorizzati nella cache solo le risposte ai **GET** (non SSL), non i POST ecc.
- Due tipi di cache
 - **Lato client** (ad esempio nel browser)
 - **Proxy/gateway/CDN**
- Le architetture SOA di grandi dimensioni utilizzano un server proxy di caching (memcache, et similia)
 - Ricordate che possono esserci cache lungo tutto il percorso tra il client e il server.

Caching locale

Un *GET condizionale* è una richiesta HTTP GET che può restituire una risposta HTTP 304 (invece di HTTP 200).
Una risposta HTTP 304 indica che la risorsa non è stata modificata rispetto alla precedente GET e quindi la risorsa non viene restituita al client in tale risposta.



Utilizzare HTTP per costruire applicazioni REST

- Header importanti per il cache-control (nella risposta del server)

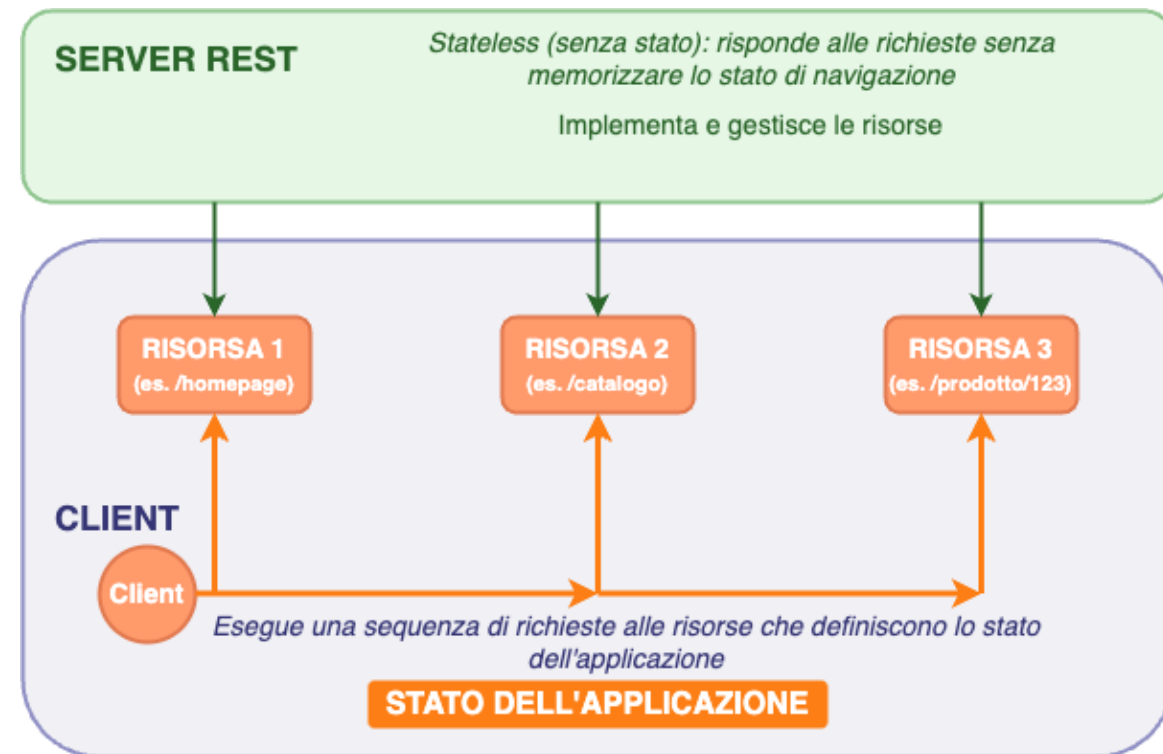
Intestazione	Proprietà
Expires	Data HTTP. Mantenere in cache fino alla scadenza
Cache-Control: max-age	Un secondi. Mantenere in cache per il tempo indicato
Cache-Control: s-maxage	Secondi. Come sopra, ma solo per i proxy
Cache-Control: public	La risorsa può essere salvata in una cache condivisa
Cache-Control: no-cache	La risorsa può essere memorizzata nella cache, ma deve essere convalidata dal server di origine prima di ogni riutilizzo
Cache-Control: no-store	Non memorizzare nella cache
Cache-Control: must-revalidate	La risorsa può essere memorizzata in cache e riutilizzata quando è "fresca". Se diventa stantia (stale), deve essere convalidata con il server di origine prima di essere riutilizzata.
Cache-Control: proxy-revalidate	Come sopra, ma solo per i proxy

Stato dell'applicazione

Nel contesto REST, lo **stato dell'applicazione** (in inglese *application state*) rappresenta il **"percorso" del client nel sistema**: la sequenza di risorse e interazioni attraversate durante l'uso dell'applicazione.

- È **gestito dal client**, non dal server.
- Include le risorse richieste, i link seguiti, i dati ricevuti o modificati.
- Si evolve **man mano che il client invia richieste** e riceve risposte che includono link ad altre risorse (principio di *Hypermedia as the Engine of Application State*, o **HATEOAS**).

Da non confondere con lo **stato del server**: lo stato interno delle risorse (es. il contenuto di un profilo utente).



Hypermedia control - Controllo ipermediale

HATEOAS: Hypermedia As The Engine Of Application State

- L' Hypermedia come motore dello stato dell'applicazione
- Manipolazione dello stato dell'applicazione = lo stato corrente manipolato dal client
- Hypermedia = link
- Seguire ***un collegamento può essere visto come una transizione di stato dell'applicazione***
 - Il server fornisce una nuova rappresentazione; il client assume tale stato.
- Il server "guida" il client verso nuovi stati fornendo collegamenti all'interno di rappresentazioni di risorse.
 - Tutte le rappresentazioni devono contenere dei link
 - Niente link, niente Web

Hypermedia control

- I client devono seguire i link forniti dal server.
 - I link dovrebbero essere "classificati" con un significato semantico, in modo che un client intelligente sappia a cosa punta il link.
 - Aiuta anche ad ridurre l'accoppiamento tra client e server.
 - I link permettono
 - **scoperta delle funzionalità**
 - **l'ordine delle interazioni** (ad esempio, come presentato)
 - **indipendenza dalla posizione (nell'albero delle API) e sicurezza**

HTTP/1.1 200 OK

Content-Type: application/vnd.acme.account+json

Content-Length: ...

```
{
  "account": {
    "numero_conto": 12345,
    "saldo": {
      "valuta": "USD",
      "valore": 100.00
    },
    "_links": {
      "self": {
        "href": "/accounts/12345"
      },
      "movimenti": {
        "href": "/accounts/12345/transactions"
      },
      "richieste_operazioni": {
        "href": "/accounts/12345/operation-requests"
      },
      "chiusura_richiesta": {
        "href": "/accounts/12345/closure-request"
      }
    }
  }
}
```


Esempio: Gestione del curriculum universitario

- **Stessi dati -> Rappresentazioni diverse:** per l'amministrazione o per la didattica
 - Rappresentazioni orientate alla macchina (XML/JSON) - rappresentazioni orientate all'uomo (HTML)
- Link definiscono le possibili azioni di manipolazione delle risorse (identificate da URI). Esempi di endpoint:

→ **GET *unimib.it/{matricola}***: restituisce informazioni sullo studente e l'elenco di altre risorse collegate:

- *nome, cognome, data-di-nascita, etc.*
- *unimib.it/{matricola}/badges*
- *unimib.it/{matricola}/pianostudio*
- *unimib.it/{matricola}/storicorate*

Solo URI, perché mancano i verbi?

→ **GET *unimib.it/{matricola}/pianostudio***:

- GET per ottenere l'elenco degli esami a cui lo studente è iscritto,
- POST per iscriversi a un esame,

→ **GET *unimib.it/{matricola}/pianostudio/{codice}***:

- GET per ottenere informazioni sull'esame a cui lo studente è iscritto
- PUT per modificare la registrazione dell'esame,
- DELETE per eliminare la registrazione dell'esame

Esempio: Gestione del curriculum universitario

→ **GET *unimib.it/{matricola}***: restituisce informazioni sullo studente e l'elenco di altre risorse collegate:

- *nome, cognome, data-di-nascita, etc.*
- *unimib.it/{matricola}/badges*
- *unimib.it/{matricola}/pianostudio*
- *unimib.it/{matricola}/storicorate*

HTTP/1.1 200 OK

Content-Type: application/vnd.unimib.student+json

Content-Length: ...

```
{
  "studente": {
    "matricola": "123456",
    "nome": "Mario",
    "cognome": "Rossi",
    "data_di_nascita": "2000-04-15",
    "email": "mario.rossi@unimib.it",
    "_links": {
      "self": {
        "href": "unimib.it/123456"
      },
      "badges": {
        "href": "unimib.it/123456/badges"
      },
      "piano_studio": {
        "href": "unimib.it/123456/pianostudio"
      },
      "storico_rate": {
        "href": "unimib.it/123456/storicorate"
      }
    }
  }
}
```

Sezione 3

COSTRUIRE APPLICAZIONI REST

REST: semplicità e standard

REST mette al centro la semplicità

- **La cache** migliora i tempi di risposta e riduce il carico del server
- **L'assenza di stato e la riduzione delle comunicazioni** facilitano il bilanciamento del carico tra i server.
- **Software meno specializzato** perché le tecnologie sottostanti sono ben note e semplici
- **L'identificazione delle risorse** avviene tramite meccanismi **standard**, non sono necessari nomi aggiuntivi.

REST è (basato su) standard

- I principi REST enfatizzano l'**uso corretto e completo del protocollo HTTP** per pubblicare servizi sul Web.
- REST+HTTP fornisce un **meccanismo leggero e stratificato** per l'integrazione di dati e servizi.
- REST+HTTP fornisce una **piattaforma applicativa distribuita e guidata dagli ipermedia**.

Utilizzare HTTP per costruire applicazioni REST

- La ricetta REST:
 - Definire i sostantivi (endpoint)
 - Definire i formati (rappresentazioni delle risorse)
 - Scegliere le operazioni
 - Evidenziare i codici di ritorno

Definire gli endpoint

- Tutto in un sistema REST è una risorsa, un **nome**.
 - Se vi ritrovate a creare dei verbi, modificateli in nomi (ApproveExpense → ExpenseApprovals).
 - Ogni risorsa ha un **URI**. Idealmente solo uno (ma non siate talebani)
- Gli URI devono essere **descrittivi**
 - <http://example.com/expenses>
 - Perdeteci del tempo, ma non agonizzate per trovare l'URI perfetto.
- Gli URI dovrebbero essere **opachi** (<https://www.w3.org/DesignIssues/Axioms.html#opaque>)
 - i client automatizzati (non umani) non dovrebbero dedurre i metadati da un URI
 - Non divulgare le informazioni sulla piattaforma /**spese**.php/123
- Gli URI dovrebbero essere mantenuti **invariati** - persistenti (<http://www.w3.org/Provider/Style/URI>)
 - Gli URI non cambiano (**semplicità, stabilità, gestibilità**)
 - Requisito fondamentale del web semantico

Utilizzare HTTP per costruire applicazioni REST

- Utilizzare le **path variables** per codificare la gerarchia
 - `/spese/pendenti/123` (`/spese/pendenti/{id}`)
- Usare altri segni di punteggiatura per **evitare di sottintendere una gerarchia**.
 - `/spese/Q107;Q307`
 - `/spese/lacey,peter`
- Usare le **query parameter per implicare le condizioni di filtraggio**
 - Le query parameter non sono così necessarie come si pensa
 - `/expenses?start=20070101&end=20071231`
dovrebbe essere
`/expenses/20070101-20071231`
 - Le cache tendono a ignorare gli URI con variabili di query

Definire i formati

- Né HTTP né REST impongono un'unica rappresentazione dei dati.
 - [Una risorsa può avere diverse rappresentazioni](#)
XML, JSON, binario (ad esempio, jpeg), coppie nome/valore
- Evitare di creare rappresentazioni personalizzate: utilizzare tipi di media ben noti (tipi MIME registrati dalla IANA)
 - Rende i contenuti più accessibili
 - Parte del vincolo di messaggistica autodescrittiva
- Le rappresentazioni in entrata devono essere uguali a quelle in uscita.
 - Un client deve essere in grado di accedere (GET) e modificare (PUT) un documento.
 - [Il server deve scartare tutti i dati estranei](#)

Esempio di applicazione

- Esempio di applicazione: Un **semplice sistema di rendicontazione delle spese**
 - Obiettivo: progettare un'API Web che esponga tutti gli elementi di un sistema di rendicontazione delle spese.
 - Gli utenti possono *creare*, *modificare* ed *eliminare* intere note spese e singole voci.

Esempio di applicazione

- Trovare tutti i nomi
 - Tipicamente espresso come modelli di URI `baseURI/{expense-id}`
- Elenco iniziale delle risorse
 - Assume l'URI di base, ad esempio `http://expenses.example.com/v1.`

Risorse	URI
Le note spese di Bill	<code>/users/bill/expenses</code>
Relazione sulle spese #123	<code>/users/bill/expenses/123</code>
Tutte le note spese (che siete autorizzati a vedere)	<code>/expenses/</code>
Tutte le note spese in sospeso	<code>/expenses/pending</code>
Le note spese in sospeso di Bill	<code>/users/bill/expenses/pending</code>
2022 spese	<code>/2022/expenses/</code>
2022 spese aperte	<code>/2022/expenses/submitted</code>

Scegliere le operazioni

- Scegliere le operazioni che possono essere applicate alle risorse
 - L'HTTP ha un'interfaccia utente limitata (insieme di verbi/operazioni/metodi).
- Per la maggior parte delle applicazioni, i metodi di base di HTTP sono sufficienti.
 - **GET** : Recupero di una risorsa (non ci devono essere effetti collaterali)
 - **POST** : Aggiunge una nuova risorsa al server.
 - **PUT** : trasferisce una risorsa a un server (sovrascrivendo se ne esiste già una) - Aggiornamento
 - **DELETE** : Scarta una risorsa
- Sono disponibili altri metodi
 - **HEAD** come GET ma senza body
 - **OPTIONS** (non ampiamente supportate)
 - **PATCH** applica un aggiornamento parziale alla risorsa (è necessario inviare solo i dati che si desidera aggiornare).
 - **TRACE** (non significativo)
 - **CONNECT** (non significativo)
- Tutte le nostre risorse supporteranno GET

Metodi HTTP

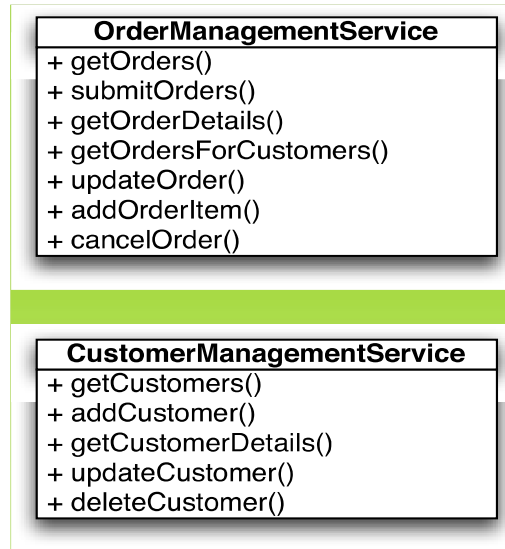
		cache	safe	idempotente
OPTIONS	rappresenta una richiesta di informazioni sulle opzioni di comunicazione disponibili sulla catena richiesta/risposta identificata dal Request-URI	✓	✓	✓
GET	significa recuperare qualsiasi risorsa identificata dall'indirizzo Request-URI	✓	✓	✓
HEAD	identico a GET, tranne per il fatto che il server NON DEVE restituire un message-body nella risposta.	✓	✓	✓
POST	è utilizzato per richiedere che il server di origine accetti l'entità contenuta nella richiesta come nuovo subordinato della risorsa identificata dal Request-URI nella Request-Line			
PUT	richiede che la risorsa allegata sia memorizzata nell'ambito del Request-URI fornito			✓
DELETE	richiede che il server di origine cancelli la risorsa identificata dal Request-URI			✓
TRACE	è usato per invocare un loop-back remoto, a livello di applicazione, del messaggio di richiesta		✓	✓
Patch	Viene utilizzato per eseguire aggiornamenti parziali su una risorsa.			Può essere

Safe = Un metodo HTTP è safe (**sicuro**) se non altera lo stato del server. In altre parole, un metodo è sicuro se porta a un'operazione di sola lettura.

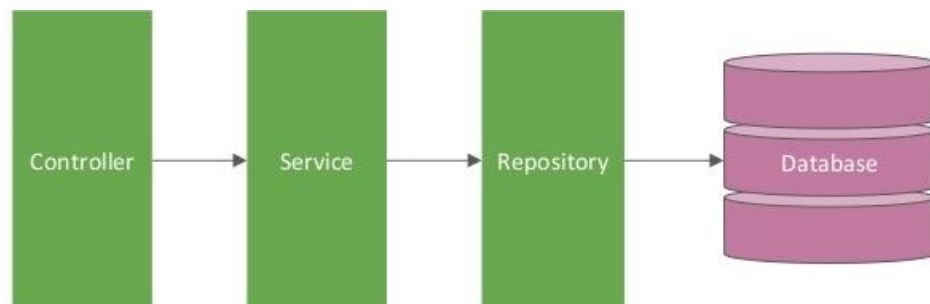
Idempotente = gli effetti collaterali di $N > 0$ richieste identiche è lo stesso di una singola richiesta

REST come implementazione CRUD

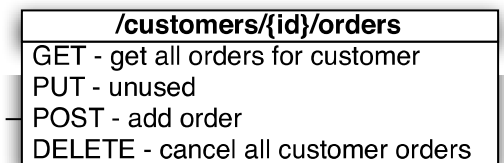
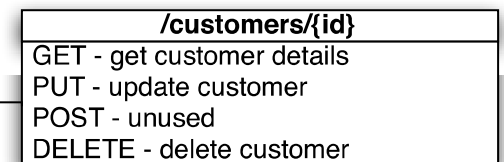
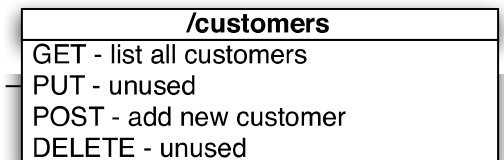
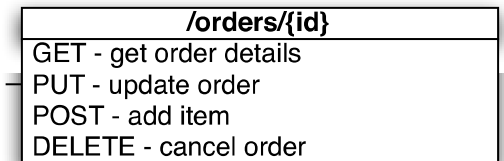
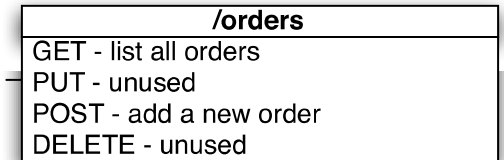
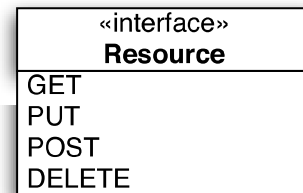
- Un esempio (stile CRUD)



Classi di servizio



Interfaccia controller REST



PUT vs. POST

Quando si creano nuove risorse

- Utilizzare **POST** se il server sceglie l'URI (l'id)
- Usare **PUT** se il cliente sceglie l'URI (l'id)

POST può essere utilizzato come "**Process this**".

- POST fa "qualcosa" e restituisce "qualcosa".
 - Di solito maschera una **chiamata di procedura remota** (RPC) (**overloaded POST**) - invocazione di funzione
- Non usarlo con troppa libertà

"Process this" POST a volte prezioso:

- Creare risorse
- A volte è necessario avere un verbo
- Se ci si trova a creare risorse che non possono essere recuperate con GET, si deve rivedere il progetto
http://expense.example.com/pay_expense
- Ma non c'è da preoccuparsi, se vi fa uscire da un impasse progettuale

HTTP/1.1 200 OK

Content-Type: application/vnd.acme.account+json

Content-Length: ...

```
{
  "account": {
    "numero_conto": 12345,
    "saldo": {
      "valuta": "USD",
      "valore": 100.00
    },
    "_links": {
      "self": {
        "href": "/accounts/12345"
      },
      "movimenti": {
        "href": "/accounts/12345/transactions"
      },
      "richieste_operazioni": {
        "href": "/accounts/12345/operation-requests"
      },
      "chiusura_richiesta": {
        "href": "/accounts/12345/closure-request"
      }
    }
  }
}
```

Codice di stato della risposta

- Il codice di stato della risposta viene generato dal server per indicare l'esito di una richiesta.
- Il codice di stato è un **numero di 3 cifre**:
 - 1xx (informativo): Richiesta ricevuta; il server sta continuando il processo.
 - 2xx (successo): Richiesta ricevuta, compresa, accettata e servita.
 - 3xx (reindirizzamento): è necessario intraprendere ulteriori azioni per completare la richiesta.
 - 4xx (errore del cliente): La richiesta contiene una sintassi errata o non può essere compresa.
 - 5xx (errore del server): Il server non è riuscito a soddisfare una richiesta apparentemente valida.
- La corretta identificazione dei codici di stato associati a una richiesta è una decisione cruciale nello sviluppo REST.

Esempio di applicazione

URI	Metodo	Operazioni commerciali	Codici di ritorno
/users/{userId}/expenses	POST	Creare una nuova nota spese	201 Created 400 Bad Request 409 Conflict 404 (User) not found 415 Unsupported media type
/users/{userId}/expenses/123	POST PUT DELETE	Aggiungere elementi nella nota spese Aggiornare la nota spese Eliminare la nota spese e le voci di spesa	201 400, 409, 404, 415 204 No Content

Sezione 4

ARGOMENTI AVANZATI

Negoziazione dei contenuti

- Una singola risorsa può avere più rappresentazioni
 - XML, PNG, JSON, ecc.
 - Inglese, giapponese, ecc.
- Il cliente può richiedere una particolare rappresentazione tramite le intestazioni della richiesta

```
Accetta: text/xml, application/xml, application/xhtml+xml,  
text/html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5
```

```
Accept-Language: en-us,en;q=0.5
```

```
Codifica di accettazione: gzip,deflate
```

```
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7 "q" = relative preference (0-1)
```

Operazioni asincrone

- Esempio: elaborazione di un assegno
 - Elaborare un assegno per una spesa. Il sistema di elaborazione degli assegni è remoto. Accetta le richieste per mezzo di una coda.
 - Trucco: esporre la coda come risorsa
 - Utilizzare la risposta "202 accettato" e creare una nuova risorsa "job" per una successiva interrogazione.

```
POST /payment+queue HTTP 1.1  
Host: expenses.example.com
```

```
<representation of a payment>
```

- ▶ 202 Accepted
- ▶ **Location:** `/payment+queue/4567`

Operazioni asincrone

- **Interrogare** la risorsa **job** per conoscerne lo stato

```
GET /payment+queue/4567 HTTP 1.1
```

```
Host: expenses.example.com
```

- 200 OK
- Content-Type: text/plain
- il pagamento di 100 dollari a Pete Lacey è in coda

(in seguito)

- il pagamento di 100 dollari a Pete Lacey è in coda

(in seguito)

Assegno di 100 dollari versato a Pete Lacey

Sulla mancanza di stato

- Ogni richiesta avviene in modo isolato
 - *Il client guida lo stato dell'applicazione*
 - Tutte le informazioni necessarie per l'esecuzione di una richiesta sono presenti nel body/header della richiesta.
 - Il server non utilizza le informazioni delle richieste precedenti (assenza di contesto)
- I possibili *stati* di un server *sono anch'essi risorse* e hanno URI, devono essere gestiti anche dal client
 - Le note spese in sospeso dalla 10a alla Xa
`/expenses/pending?start=10`
Può essere richiamato senza visitare la prima pagina delle spese.
Non è necessario uno stato sincronizzato, ad esempio la "directory di lavoro" di FTP.
 - Luci accese in questo momento: `/lights/currently_on`

Sulla mancanza di stato

- Il carico delle applicazioni prive di stato può essere *bilanciato, memorizzato in cache*
- Le applicazioni possono essere *partizionate, e scalate*.
- Le "sessioni" sono da evitare
 - Le sessioni sono server che mantengono lo stato lato client
 - I messaggi non possono più essere autodescrittivi
 - Il server può rispondere in modo diverso a richieste identiche in base alla sessione
- Il server dovrebbe dimenticarsi dei client tra una richiesta e l'altra
 - Nessun cookie
 - Nessuna chiave di sessione nell'URI o nei dati della richiesta
- Fare molta attenzione se si infrange il vincolo di mancanza di stato
 - Ha un impatto sull'affidabilità, la scalabilità, le prestazioni

Sezione 5

REST VS SOAP — UN CONFRONTO

REST vs SOAP

	REST	SOAP
Protocollo di trasporto	HTTP	Vari protocolli (HTTP, TCP, SMTP)
Formato del messaggio	Vari formati (XML, JSON)	XML-SOAP (envelope, header, body)
Identificatori di servizio	URI	URI e indirizzamento WS
Descrizione del servizio	OpenAPI-Swagger, documentazione testuale	WSDL (Web Services Description Language)
Service Discovery	Nessun supporto standard	UDDI
Flessibilità	Più leggero e flessibile	Più rigido ma con maggiore standardizzazione
Stato	Stateless	Può mantenere stato
Prestazioni	Generalmente più veloce e con meno overhead	Maggiore overhead dovuto alla struttura XML